

Interpretabilidad indirecta ANC a través de Tracr-RASP

JUAN RODRIGO ANABALÓN RIQUELME, Universidad Nacional de Cuyo, Argentina

JUAN B. CABRAL, Universidad Nacional de Córdoba, Argentina

Las redes de Criptografía Neuronal Adversarial (ANC, Abadi and Andersen [1]) aprenden a cifrar mensajes de extremo a extremo mediante un juego minimax entre tres redes (Alice, Bob, Eve), sin que el algoritmo de cifrado resultante sea legible a partir de los pesos entrenados. En este trabajo proponemos usar un transformer compilado con Tracr [4] —que implementa exactamente un cifrado XOR bit a bit a partir de un programa RASP conocido [8]— como fuente de un vocabulario de variables interpretables (bit de clave alineado por posición, bit de salida XOR por posición) que sirve de hipótesis para sondear linealmente la única capa oculta de una red Alice (un MLP) entrenada de forma opaca. El sondeo revela una heterogeneidad invisible en la métrica agregada de coincidencia con XOR: algunos bits de salida son perfectamente legibles de forma lineal, mientras que otros permanecen a nivel de azar. Mostramos que esta heterogeneidad coincide exactamente con el colapso de la norma de los pesos de salida correspondientes, consistente con un óptimo local del objetivo adversarial. Mediante una ablación controlada (barrido de los pesos de la función de pérdida y del ancho de la capa oculta, 5 semillas por configuración) mostramos que el fenómeno es principalmente un cuello de botella de capacidad de la arquitectura original de Abadi y Andersen, y no una limitación fundamental del esquema de entrenamiento: duplicar el ancho de la capa oculta reduce el colapso de 2.60 ± 1.02 a 0.40 ± 0.49 posiciones colapsadas (de 8) en promedio, y combinado con un reajuste de los pesos de la pérdida lo elimina por completo ($\text{xor_match} = 1.000$, 0 posiciones colapsadas en las 5 semillas). El trabajo ilustra un uso concreto de modelos compilados como generadores de hipótesis interpretativas para diagnosticar patologías de entrenamiento en redes opacas.

Additional Key Words and Phrases: interpretabilidad mecánica, Tracr, RASP, criptografía neuronal adversarial, sondeo lineal, transformers compilados

1 Introducción

Abadi y Andersen [1] mostraron que tres redes neuronales entrenadas adversarialmente —Alice, Bob y Eve— pueden aprender, sin que se les prescriba ningún algoritmo criptográfico, una forma de cifrado que Bob puede invertir usando una clave compartida y que Eve, sin acceso a la clave, no puede. El resultado es notable precisamente porque es *end-to-end*: el cifrado que emerge en los pesos de Alice no es legible directamente, y los propios autores reportan que, en su formulación original (sin ningún término adicional que lo empuje hacia una forma particular), el cifrado emergente *no* es simplemente XOR: un cambio en un bit de clave o de mensaje se difunde de forma continua sobre varios elementos del texto cifrado, en lugar de afectar un único bit como lo haría un XOR exacto. Para poder comparar directamente contra una referencia con semántica conocida, en este trabajo agregamos al objetivo de Alice un término auxiliar que explícitamente empuja el cifrado hacia la forma XOR bit a bit (Sección 3). Esto no está garantizado por el objetivo adversarial en sí, sino que es una elección de diseño que hace que XOR sea el blanco declarado del entrenamiento; pero nada garantiza que ese blanco se alcance de manera completa o uniforme a lo largo de toda la salida —y es precisamente esa brecha la que investigamos.

Esto plantea un problema de interpretabilidad concreto: ¿cómo se verifica, más allá de una métrica de precisión agregada, si una red entrenada de forma opaca efectivamente implementa la operación que se espera que aprenda? Los métodos de interpretabilidad post-hoc habituales (mapas de saliencia, LIME, SHAP) no fueron diseñados para responder preguntas de esta forma —no hay, a priori, ninguna referencia contra la cual comparar la explicación producida.

Authors' Contact Information: Juan Rodrigo Anabalón Riquelme, misc@monkeyslab.cl, Universidad Nacional de Cuyo, Argentina; Juan B. Cabral, Universidad Nacional de Córdoba, Argentina.

Tracr [4] ofrece una solución indirecta a este problema: permite escribir un programa en RASP [8] —un lenguaje simbólico para razonar sobre computación en transformers— y compilarlo en un transformer real cuyos pesos implementan *exactamente* ese programa, con una estructura interna conocida por construcción (variables del residual stream etiquetadas, patrones de atención predecibles). El objetivo original de Tracr es servir de “laboratorio”: un modelo con ground-truth interno conocido sobre el cual validar técnicas de interpretabilidad antes de aplicarlas a modelos opacos.

En este trabajo llevamos esa idea un paso más allá de su uso habitual. En lugar de usar el modelo compilado únicamente para validar una técnica de interpretabilidad en abstracto, usamos el *vocabulario de variables* que expone Tracr para un programa RASP de XOR —concretamente, la noción de “bit de clave traído a la posición del mensaje” y “bit de salida XOR en la posición i ”— como hipótesis concreta para sondear una red Alice entrenada de forma opaca. Como Alice es un MLP y el modelo compilado es un transformer, la comparación no puede hacerse a nivel de circuito o de neurona a neurona; el punto de contraste es conceptual: si Alice aprendió efectivamente una función equivalente a XOR, debería existir, en algún lugar de su representación interna, una codificación linealmente legible del bit de salida por posición, análoga a la variable categórica que Tracr expone de forma explícita.

1.1 Pregunta de investigación y contribución

Nuestra pregunta de investigación es: *¿el vocabulario de variables de un modelo compilado con Tracr puede usarse como generador de hipótesis para un sondeo interpretativo de una red adversarial entrenada de forma opaca, y ese sondeo revela algo que las métricas agregadas de desempeño no muestran?* Encontramos que la respuesta es afirmativa, y que además el sondeo lineal permite diagnosticar mecánicamente una patología de entrenamiento —un colapso de peso por posición de salida— que resulta ser, mayormente, un problema de capacidad de la arquitectura y no una limitación intrínseca del esquema ANC. Las contribuciones concretas de este trabajo son:

- (1) Una metodología de sondeo lineal por posición, usando como target las variables categóricas expuestas por un programa RASP compilado, aplicada a la única capa oculta de una red Alice (Sección 3).
- (2) La identificación de una heterogeneidad no uniforme en la convergencia de Alice hacia XOR, invisible en la métrica agregada usual, y su explicación mecánica en términos de la norma de los pesos de salida por posición (Sección 4).
- (3) Una ablación controlada que aísla el efecto de los pesos de la función de pérdida adversarial (α , β) del efecto de la capacidad de la capa oculta, mostrando que ambos contribuyen pero que la capacidad domina el fenómeno (Sección 4).

2 Trabajo relacionado

Criptografía neuronal adversarial. Abadi and Andersen [1] introdujeron el esquema Alice–Bob–Eve entrenado mediante un objetivo minimax en el que Alice minimiza el error de reconstrucción de Bob y es recompensada cuando el error de Eve se aleja del valor esperado por azar. Los propios autores reportan que la forma de cifrado que emerge de ese objetivo, sin ninguna presión adicional, generalmente *no* es un XOR bit a bit sino una función continua que difunde cada bit de entrada sobre varios elementos del texto cifrado; también reportan que el entrenamiento adversarial no siempre converge (hasta un tercio de las corridas fallan según sus propios experimentos), sin diagnosticar la causa a nivel de pesos individuales. Nuestro trabajo usa una variante de su esquema con un término auxiliar explícito que sí empuja el cifrado hacia la forma XOR (Sección 3) —precisamente para tener un blanco con semántica conocida contra

el cual comparar— y retoma el problema abierto de diagnosticar por qué la convergencia a ese blanco no siempre es completa.

RASP y Tracr. Weiss et al. [8] proponen RASP como modelo computacional para razonar sobre lo que un transformer puede o no expresar en función de su número de capas y cabezas de atención. Lindner et al. [4] construyen sobre RASP un compilador (tracr) que traduce un programa RASP a los pesos de un transformer real, produciendo un modelo con estructura interna completamente conocida. Los autores proponen explícitamente usar estos modelos compilados como banco de pruebas (“laboratorio”) para técnicas de interpretabilidad, en lugar de como objeto de estudio en sí mismos —es exactamente ese marco el que extendemos aquí, usando el vocabulario del modelo compilado como fuente de hipótesis para sondear una red externa.

Sondeo lineal (probing). La idea de entrenar clasificadores lineales sobre representaciones internas de una red para evaluar si una propiedad está linealmente codificada se remonta a Alain and Bengio [2]. Usamos esa misma herramienta, pero con una diferencia relevante: en lugar de sondear por una propiedad genérica del dominio, sondeamos específicamente por las variables que un modelo con semántica conocida (Tracr) ya demostró que son necesarias y suficientes para resolver la tarea.

Interpretabilidad mecanística. Olah et al. [7] argumentan que los mecanismos internos de una red pueden entenderse estudiando las conexiones entre neuronas como circuitos. Olah [6] discuten la importancia de que la base de variables que usa una red para representar un problema coincida con una base interpretable por humanos —la interpretabilidad, argumentan, no es solo una propiedad del método usado sino también del modelo y de si su representación natural es o no legible. Nuestro resultado puede leerse como una instancia concreta de ese argumento: la red Alice original solo tiene una base interpretable (una codificación lineal limpia por bit) en una fracción de sus posiciones de salida, y el resto queda opaco hasta que se corrige un cuello de botella de capacidad.

3 Método

3.1 Esquema ANC

Usamos una versión simplificada del esquema de tres redes de Abadi y Andersen. La arquitectura original combina una capa totalmente conectada con varias capas convolucionales 1D (“mix & transform”); en cambio, tanto Alice como Bob aquí son un MLP de una sola capa oculta, elegido deliberadamente así porque el objeto de estudio de este trabajo es esa capa oculta. Con longitud de mensaje N , Alice recibe la concatenación de mensaje y clave ($2N$ bits), la pasa por una capa oculta totalmente conectada de tamaño `hidden_size` con activación ReLU, y produce un texto cifrado de N bits mediante una capa de salida con activación sigmoide. Bob tiene la misma arquitectura, recibiendo cifrado y clave. Eve, sin acceso a la clave, recibe solo el cifrado. Por defecto `hidden_size` = $2N$ (el mismo tamaño que la entrada concatenada); en la Sección 4 tratamos `hidden_size` como un hiperparámetro independiente para aislar su efecto.

El objetivo de entrenamiento también es una variante del de Abadi y Andersen, que usan una penalización cuadrática sobre qué tan lejos está el error de Eve de lo esperado por azar y no incluyen ningún término de XOR explícito. Nuestra variante combina tres términos:

$$\mathcal{L}_{\text{Alice}} = \mathcal{L}_{\text{Bob}} - \alpha \mathcal{L}_{\text{Eve}} + \beta \mathcal{L}_{\text{XOR}} \quad (1)$$

donde $\mathcal{L}_{\text{Bob}}$ y $\mathcal{L}_{\text{Eve}}$ son la entropía cruzada binaria entre la predicción de cada decodificador y el mensaje original, y $\mathcal{L}_{\text{XOR}}$ es la entropía cruzada entre el texto cifrado producido y la referencia XOR explícita bit a bit —el término que garantiza que exista un blanco conocido contra el cual comparar, ausente en la formulación original—. α pondera cuánto se premia confundir a Eve; β pondera cuánto se empuja el cifrado hacia la forma XOR específicamente. Cada paso de

entrenamiento actualiza primero Bob, luego Alice, y finalmente Eve durante varias sub-iteraciones (6 por defecto) para mantener a Eve cerca de su óptimo dado el Alice actual, siguiendo el esquema de entrenamiento adversarial estándar. Usamos Adam [3] con tasa de aprendizaje 10^{-3} para las tres redes.

3.2 Modelo de referencia compilado con Tracr

Implementamos el cifrado XOR como un programa RASP: la entrada es una única secuencia de largo $2N$, con los bits del mensaje en las posiciones $[0, N)$ y los de la clave en $[N, 2N)$. Para alinear cada bit de mensaje con su bit de clave correspondiente se usa un mecanismo de atención expresado como Select/Aggregate sobre los índices de posición (`key_pos == query_pos + N`), que “trae” categóricamente el valor del bit de clave a la posición del bit de mensaje; un `SequenceMap` calcula entonces el XOR bit a bit entre ambos valores. Compilado con `tracr.compiler.compiling.compile_rasp_to_model`, el resultado es un transformer mínimo: una capa, una cabeza de atención, `key_size=10`, `mlp_hidden_size=4`, con un residual stream de dimensiones categóricas etiquetadas (`model.residual_labels`). Dos de esas etiquetas son el vocabulario que usamos como hipótesis en el resto del trabajo: `aggregate_2: {0, 1}` (el bit de clave traído a la posición del mensaje) y `sequence_map_1: {0, 1}` (el bit de salida XOR en esa posición). Verificamos programáticamente, leyendo los logits de atención del modelo compilado (no el código RASP), que cada una de las N posiciones de mensaje efectivamente atiende a la posición `pos+N` ($N = 4$ en nuestra verificación). Como el mecanismo de Select de este programa RASP depende únicamente de índices de posición y no de los valores de los bits, el patrón de atención es idéntico para cualquier mensaje y clave —una sola verificación alcanza para confirmar el mecanismo en general, y este es legible directamente en los pesos, sin necesidad de inspeccionar el código RASP fuente.

3.3 Sondeo lineal guiado por el vocabulario de Tracr

Dado que Alice es un MLP de una sola capa oculta y Tracr compila un transformer, la comparación no puede hacerse circuito a circuito. En cambio, tomamos la variable `sequence_map_1` (el bit XOR de salida en la posición i) como hipótesis y preguntamos: ¿está esa misma información codificada de forma linealmente legible en la capa oculta de Alice? Para cada posición de salida $i \in [0, N)$ entrenamos una sonda lineal (regresión logística, implementada como una capa `nn.Linear` entrenada con `BCEWithLogitsLoss` y Adam, sin dependencias externas de *probing*) que predice el bit XOR de esa posición a partir de dos posibles fuentes de features:

- **input** (control): la concatenación cruda de mensaje y clave. Como el valor de XOR en una posición no es linealmente separable en función de únicamente esos dos bits relevantes —el problema clásico de separabilidad de XOR [5]—, este control acota por debajo lo que puede lograr una sonda lineal si Alice no realizó ningún cómputo no lineal útil.
- **hidden**: las activaciones post-ReLU de la única capa oculta de Alice (`fc1`), el análogo funcional del residual stream de Tracr.

Cada sonda se entrena con 4096 ejemplos y se evalúa sobre 1024 ejemplos held-out generados de manera independiente.

3.4 Diagnóstico de colapso y ablaciones

Al observar que la accuracy de la sonda hidden es marcadamente bimodal entre posiciones (cercana a 1.0 o cercana al azar, rara vez intermedia), inspeccionamos la norma de cada fila de la matriz de pesos `fc2.weight` de Alice (la conexión capa oculta $\rightarrow$ salida), una fila por posición de salida. Definimos una posición como *colapsada* si esa norma cae por

debajo de un umbral de 4.0. El umbral se eligió post-hoc, pero está bien justificado empíricamente: agrupando las normas de fila observadas en los 280 pares (posición, configuración) de las ablaciones de la Sección 4, existe un hueco limpio entre 3.68 (el valor más alto por debajo del umbral) y 4.25 (el valor más bajo por encima), sin ninguna observación en ese intervalo.

Para investigar la causa del colapso realizamos dos barridos, cada uno con 5 semillas aleatorias fijas (0–4) y 10 000 pasos de entrenamiento, $N = 8$, tamaño de minibatch 128:

- (1) **Barrido de pesos de pérdida:** 7 combinaciones de $(\alpha, \beta) = (1, 0.5), (1, 0.1), (2, 0.5), (2, 0.1), (3, 0.5), (2, 1.0)$ y $(3, 1.0)$ — con `hidden_size` fijo en $2N = 16$ (arquitectura original).
- (2) **Barrido de capacidad:** `hidden_size` $\in \{16, 32, 64, 128\}$, con (α, β) fijo en la mejor combinación encontrada en el barrido anterior $(3.0, 1.0)$, más una verificación de aislamiento cruzando `hidden_size` $\in \{16, 32\}$ con $(\alpha, \beta) \in \{(1.0, 0.5), (3.0, 1.0)\}$ para separar el efecto de la capacidad del efecto del balance de pérdidas.

En cada configuración medimos el `xor_match` promedio (fracción de bits de salida que coinciden con la referencia XOR explícita, promediada sobre posiciones y sobre un batch de evaluación de 4096 ejemplos) y el número de posiciones colapsadas según el criterio de norma anterior.

4 Resultados

4.1 El sondeo revela heterogeneidad invisible en la métrica agregada

Con $N = 8$, $\alpha = 1.0$, $\beta = 0.5$ (valores por defecto) y 22 000 pasos de entrenamiento (semilla 0), el `xor_match` agregado se estabiliza en ≈ 0.83 , sin llegar nunca a 1.0. La Tabla 1 muestra la accuracy de la sonda lineal por posición para las dos fuentes de features.

Table 1. Accuracy de la sonda lineal (test set) por posición de salida, $N = 8$, semilla 0, 22 000 pasos de entrenamiento.

Posición	0	1	2	3	4	5	6	7
input (control)	0.501	0.485	0.467	0.529	0.479	0.521	0.503	0.515
hidden (Alice)	1.000	0.500	0.521	1.000	0.634	1.000	1.000	1.000

El control input se mantiene, como se predijo, cercano al azar en las 8 posiciones —el límite teórico de separabilidad lineal para XOR. La sonda hidden, en cambio, es marcadamente bimodal: 5 de 8 posiciones (0, 3, 5, 6, 7) son legibles con accuracy perfecta, tan limpiamente como la variable categórica `sequence_map_1` en el modelo compilado con Tracr, mientras que las posiciones 1 y 2 quedan al nivel de azar y la posición 4 es intermedia. Esta heterogeneidad es consistente aritméticamente con el `xor_match` agregado observado: si 5 posiciones están resueltas perfectamente y las 3 restantes al azar, el promedio esperado es $(5 \times 1.0 + 3 \times 0.5)/8 \approx 0.8125$, casi exactamente el valor medido. Es decir, las posiciones donde la sonda lineal lee el bit XOR con perfección son las mismas donde Alice efectivamente convergió al cifrado correcto.

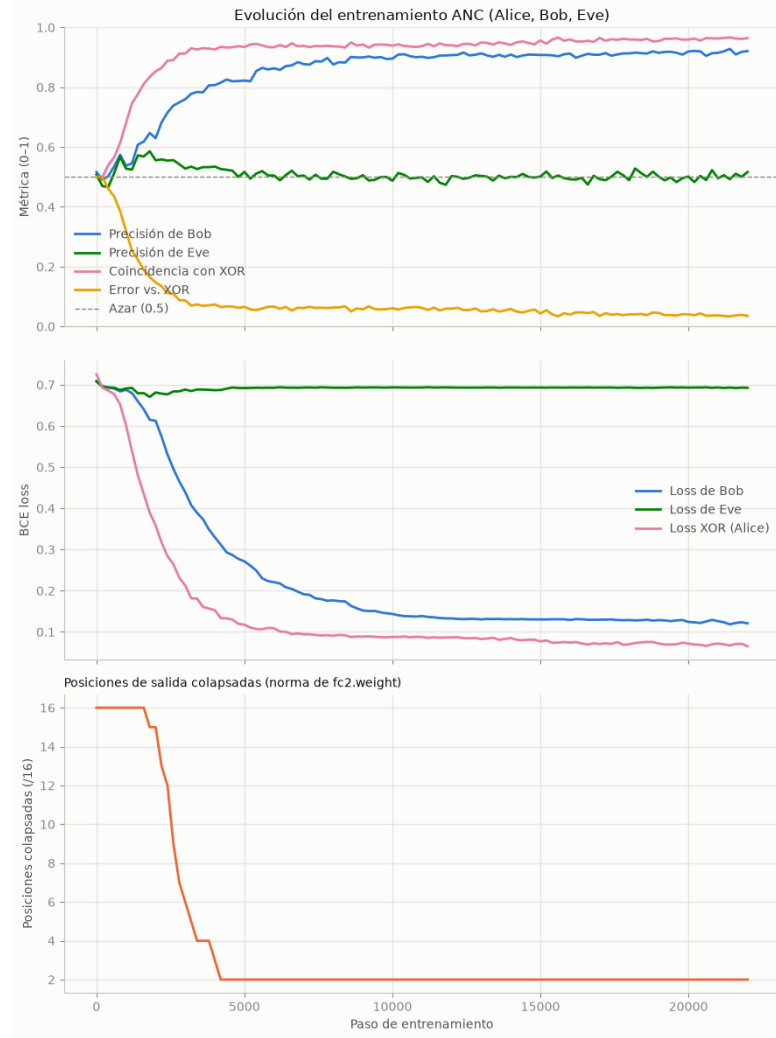


Fig. 1. Dinámica de entrenamiento de una corrida representativa ($N = 16$, $\alpha = 1.0$, $\beta = 0.5$, 22 000 pasos, semilla 0; distinta de las corridas $N = 8$ usadas en las ablaciones de esta sección). De arriba a abajo: precisión de Bob/Eve y coincidencia/error con XOR; el loss de evaluación de cada red; y el número de posiciones de salida colapsadas (norma de fila de `fc2.weight` por debajo del umbral). El colapso cae de 16/16 a un piso de 2/16 a medida que `xor_match` converge, ilustrando a nivel de una sola corrida el mismo mecanismo que la Tabla 1 muestra a nivel de posición.

4.2 El colapso es consistente entre semillas y se explica por la norma de los pesos de salida

Repetimos el entrenamiento con 5 semillas distintas (10 000 pasos, $\alpha = 1.0$, $\beta = 0.5$) y medimos, para cada una, el `xor_match` de cada posición individual y la norma de la fila correspondiente de `fc2.weight` —40 pares (norma, `xor_match`) en total. La correlación de Pearson entre ambas cantidades es $r = 0.97$. La relación no es un escalón perfecto entre azar y perfección: hay casos intermedios genuinos en ambos extremos del umbral (p. ej. norma = 3.68 $\Rightarrow$ `xor_match` = 0.872; norma = 6.05 $\Rightarrow$ `xor_match` = 0.875), pero el patrón es inequívoco —dentro del grupo con norma

por debajo del umbral de 4.0 definido en la sección anterior, el `xor_match` por posición varía entre 0.481 y 0.872 (media 0.559); dentro del grupo con norma por encima del umbral, varía entre 0.875 y 1.000 (media 0.991), sin superposición entre ambos rangos. Ninguna semilla presentó unidades ReLU muertas en la capa oculta (0/16 en las 5 semillas), descartando esa hipótesis alternativa: el colapso está específicamente en la conexión capa oculta $\rightarrow$ salida de la posición afectada, no en la representación oculta compartida. Qué posición específica colapsa varía con la semilla (no hay una posición estructuralmente privilegiada), pero el mecanismo es siempre el mismo.

Interpretamos este colapso como un óptimo local estable del objetivo minimax de tres redes: cuando el peso de salida de un bit colapsa hacia cero, el sigmoide converge a una salida aproximadamente constante ≈ 0.5 para ese bit, independiente del input. En ese régimen, Bob acierta ese bit solo $\approx 50\%$ del tiempo (penalizando $\mathcal{L}_{\text{Bob}}$), pero Eve también queda en $\approx 50\%$ (beneficiando el término $-\alpha\mathcal{L}_{\text{Eve}}$) —“rendirse” en un bit específico intercambia precisión de Bob por confusión de Eve, y como el target de ese bit es aleatorio 50/50, el gradiente que debería revertir el colapso se cancela en promedio, dejando el punto como un atractor estable.

4.3 El balance de la función de pérdida modula el colapso pero no lo elimina

Table 2. Barrido de pesos de pérdida (α, β) , $N = 8$, `hidden_size`= 16, 10 000 pasos, promedio $\pm$ desviación estándar sobre 5 semillas.

Configuración	<code>xor_match</code>	Posiciones colapsadas (/8)
$\alpha=1.0, \beta=0.5$ (original)	0.850 ± 0.059	2.60 ± 1.02
$\alpha=1.0, \beta=0.1$	0.500 ± 0.002	8.00 ± 0.00
$\alpha=2.0, \beta=0.5$	0.915 ± 0.032	1.60 ± 0.49
$\alpha=2.0, \beta=0.1$	0.504 ± 0.007	8.00 ± 0.00
$\alpha=3.0, \beta=0.5$	0.871 ± 0.045	2.40 ± 0.80
$\alpha=2.0, \beta=1.0$	0.916 ± 0.022	1.60 ± 0.49
$\alpha=3.0, \beta=1.0$	0.921 ± 0.014	1.60 ± 0.49

La Tabla 2 resume el barrido de pesos de pérdida. Bajar β no reduce el colapso: lo generaliza a las 8 posiciones, pero por un mecanismo distinto. Con $\beta = 0.1$, `bob_acc` sube a ≈ 0.97 –1.0 mientras `xor_match` cae al azar en las 8 posiciones —Alice deja de intentar converger a la forma XOR específicamente, y en cambio encuentra otra función (no-XOR) para comunicarse con Bob. β es entonces lo que fuerza la convergencia a la forma XOR, no un término prescindible.

Subir α manteniendo $\beta = 0.5$ sí ayuda: con $\alpha = 2.0$ el promedio de posiciones colapsadas baja de 2.60 a 1.60 y `xor_match` sube de 0.850 a 0.915. Sin embargo, subir α aún más (3.0) sin acompañarlo de un aumento de β es *peor* que $\alpha = 2.0$ (2.40 vs. 1.60 posiciones colapsadas) —el efecto no es monótono. Las tres configuraciones con mejor desempeño ($\alpha=2/\beta=0.5$, $\alpha=2/\beta=1.0$, $\alpha=3/\beta=1.0$) convergen al mismo piso de 1.60 ± 0.49 posiciones colapsadas en promedio, sugiriendo que el ajuste de estos dos hiperparámetros por sí solo, dentro del rango explorado, no elimina el fenómeno.

4.4 El colapso es principalmente un cuello de botella de capacidad

La Tabla 3 muestra el resultado más contundente del trabajo: duplicar el ancho de la capa oculta (16 $\rightarrow$ 32) elimina el colapso por completo, en las 5 semillas, sin excepción. Ampliar aún más (64, 128) no aporta ninguna mejora adicional —el techo ya se alcanza en $4N$.

Para aislar si el efecto es puramente de capacidad o requiere también el ajuste de pérdidas de la sección anterior, cruzamos `hidden_size` $\in \{16, 32\}$ con $(\alpha, \beta) \in \{(1.0, 0.5)_{\text{original}}, (3.0, 1.0)_{\text{ajustado}}\}$ (Tabla 4).

Table 3. Barrido de `hidden_size`, $N = 8$, $\alpha = 3.0$, $\beta = 1.0$, 10 000 pasos, promedio $\pm$ desviación estándar sobre 5 semillas.

<code>hidden_size</code>	<code>xor_match</code>	Posiciones colapsadas (/8)
16 (= $2N$, original)	0.921 ± 0.014	1.60 ± 0.49
32 (= $4N$)	1.000 ± 0.000	0.00 ± 0.00
64 (= $8N$)	1.000 ± 0.000	0.00 ± 0.00
128 (= $16N$)	1.000 ± 0.000	0.00 ± 0.00

Table 4. Aislamiento del efecto de capacidad vs. balance de pérdidas, $N = 8$, 10 000 pasos, promedio $\pm$ desviación estándar sobre 5 semillas.

<code>hidden_size</code>	(α, β)	<code>xor_match</code>	Colapsadas (/8)
16	original (1.0, 0.5)	0.850 ± 0.059	2.60 ± 1.02
32	original (1.0, 0.5)	0.958 ± 0.035	0.40 ± 0.49
16	ajustado (3.0, 1.0)	0.921 ± 0.014	1.60 ± 0.49
32	ajustado (3.0, 1.0)	1.000 ± 0.000	0.00 ± 0.00

Los dos efectos son reales, independientes, y se combinan. Más capacidad por sí sola (`hidden_size`= 32 con los pesos de pérdida *originales*) ya reduce el colapso de 2.60 a 0.40 posiciones y sube `xor_match` de 0.850 a 0.958 —la capacidad explica la mayor parte del fenómeno. Pero ninguno de los dos factores por sí solo alcanza el resultado perfecto: se necesita capacidad suficiente (`hidden_size` ≥ 32 , el doble de la arquitectura original) y el reajuste de (α, β) juntos para llegar a `xor_match` = 1.000 con cero posiciones colapsadas en las 5 semillas sin excepción.

5 Discusión y conclusiones

5.1 El vocabulario de Tracr como generador de hipótesis, no como comparación circuito a circuito

El resultado central de este trabajo no depende de que Alice (un MLP) y el modelo compilado (un transformer) compartan estructura de circuito —no la comparten, y no podrían compararse directamente peso a peso. Lo que transferimos de Tracr a AliceNet no es una arquitectura sino un *vocabulario de variables*: la noción de que existe, para cada posición de salida, un bit categórico (el XOR de esa posición) que en el modelo con semántica conocida es legible al 100% de forma lineal. Ese vocabulario se convierte en una hipótesis falsable sobre la red opaca (§3.3), y resulta que la hipótesis es cierta solo parcialmente —lo cual es en sí mismo el hallazgo. Esto es consistente con el argumento de Olah [6] de que la interpretabilidad de un modelo depende de si su base de representación natural coincide con una base interpretable por humanos: aquí mostramos una forma operacional y falsable de poner a prueba esa coincidencia, usando un modelo con base conocida por construcción como fuente de la hipótesis.

5.2 De síntoma agregado a diagnóstico mecánico

Sin el sondeo por posición, el único síntoma visible habría sido que `xor_match` se estanca en ≈ 0.83 – 0.92 sin llegar a 1.0, algo que admite muchas explicaciones posibles (ruido de entrenamiento, número insuficiente de pasos, mala elección de hiperparámetros del optimizador). El sondeo lineal por posición, guiado por el vocabulario de Tracr, redujo el espacio de hipótesis a una específica y verificable: hay una subred de posiciones que efectivamente colapsó. Inspeccionar la norma de `fc2.weight` confirmó el mecanismo exacto (colapso de la conexión capa oculta $\rightarrow$ salida, no de la capa oculta en sí), y la ablación de capacidad confirmó la causa raíz (capacidad insuficiente de la capa oculta compartida).

Este encadenamiento —síntoma agregado, hipótesis guiada por vocabulario interpretable, sondeo, inspección de pesos, ablación causal— es reproducible como metodología general para diagnosticar por qué una red adversarial no converge al comportamiento esperado, más allá del caso particular de XOR.

5.3 Limitaciones

Este trabajo tiene limitaciones que acotan el alcance de sus conclusiones. Primero, la tarea (XOR bit a bit, $N = 8$) es deliberadamente simple; no está claro que el mismo patrón de colapso por posición aparezca, o tenga la misma causa, en cifrados más complejos o en tareas fuera del dominio criptográfico. Segundo, el umbral de colapso (norma < 4.0) se eligió post-hoc a partir del hueco observado empíricamente en la distribución agregada de normas (§3), y no de un criterio independiente del dato; aunque la separación de norma es limpia, el `xor_match` asociado a cada lado del umbral no lo es igual de tajantemente —existen posiciones con norma justo por debajo del umbral cuyo `xor_match` ya es sustancialmente mayor que el azar (§4)—, y un umbral fijo podría no generalizar a otras escalas de N o de `hidden_size`. Tercero, el sondeo lineal establece legibilidad, no necesidad causal: no realizamos intervenciones (activation patching) dentro de la propia red Alice para confirmar que la unidad de información identificada es efectivamente usada causalmente por el resto de la red, solo inferimos causalidad indirectamente a través de la ablación de arquitectura (reentrenamientos completos con distinta capacidad). Cuarto, el número de semillas (5) por configuración, aunque suficiente para mostrar el patrón cualitativo de forma consistente, es modesto para una caracterización estadística fina de los promedios reportados.

5.4 Conclusiones

Mostramos un uso concreto del vocabulario de variables expuesto por un modelo compilado con Tracr como generador de hipótesis para un sondeo lineal aplicado a una red de criptografía neuronal adversarial entrenada de forma opaca. Ese sondeo reveló una heterogeneidad de convergencia por posición de salida invisible en la métrica agregada habitual, y permitió diagnosticar mecánicamente su causa: un colapso de la norma de los pesos de salida específico por posición, consistente con un óptimo local del objetivo minimax de tres redes. Mediante una ablación controlada mostramos que este colapso es principalmente un cuello de botella de capacidad de la arquitectura original de Abadi y Andersen —duplicar el ancho de la capa oculta lo reduce drásticamente incluso sin tocar los pesos de la función de pérdida, y combinado con un reajuste de esos pesos lo elimina por completo— y no una limitación fundamental del esquema de entrenamiento adversarial en sí. El resultado ilustra que el valor de un modelo con ground-truth compilado no se limita a validar técnicas de interpretabilidad en abstracto: su vocabulario de variables puede transferirse directamente como hipótesis de diagnóstico sobre una red opaca no relacionada arquitectónicamente, y ese diagnóstico puede guiar una corrección de diseño concreta y verificable.

Declaración de ética

Este trabajo no involucra datos personales, sujetos humanos, ni aplicaciones de doble uso más allá de las ya inherentes a la investigación en criptografía neuronal, un dominio establecido en la literatura de aprendizaje automático desde Abadi and Andersen [1]. Todos los experimentos se realizaron sobre datos sintéticos (mensajes y claves generados aleatoriamente) y modelos de juguete a pequeña escala, sin riesgos de seguridad prácticos.

Acknowledgments

Este proyecto está financiado por InsaneMonkey SpA — MonkeysLab Unhinged Research (<https://www.monkeyslab.cl>).

References

- [1] Martin Abadi and David G. Andersen. 2016. Learning to Protect Communications with Adversarial Neural Cryptography. *arXiv preprint arXiv:1610.06918* (2016).
- [2] Guillaume Alain and Yoshua Bengio. 2016. Understanding Intermediate Layers Using Linear Classifier Probes. *arXiv preprint arXiv:1610.01644* (2016).
- [3] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.
- [4] David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. 2023. Tracr: Compiled Transformers as a Laboratory for Interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2301.05062.
- [5] Marvin Minsky and Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- [6] Chris Olah. 2022. Mechanistic Interpretability, Variables, and the Importance of Interpretable Bases. Transformer Circuits Thread. <https://transformer-circuits.pub/2022/mech-interp-essay>
- [7] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. 2020. Zoom In: An Introduction to Circuits. *Distill* (2020). doi:10.23915/distill.00024.001
- [8] Gail Weiss, Yoav Goldberg, and Eran Yahav. 2021. Thinking Like Transformers. In *Proceedings of the 38th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 139)*.